

Received 2 May 2025, accepted 7 June 2025, date of publication 12 June 2025, date of current version 19 June 2025.

Digital Object Identifier 10.1109/ACCESS.2025.3579294

RESEARCH ARTICLE

System Verification and FPGA Implementation of Hardware Preemptive Scheduler for RISC-V Processor

IONEL ZAGAN¹, (Member, IEEE), AND VASILE GHEORGHÎĂ GĂITAN¹, (Member, IEEE)

PDADMCPI Laboratory, MANSiD Research Center, Stefan cel Mare University of Suceava, 720229 Suceava, Romania

Corresponding authors: Ionel Zagan (zagan@usm.ro) and Vasile Gheorghîă Găitan (vgaitan@usm.ro)

This work is also supported by the NetZeRoCities Competence Center, funded by European Union–NextGenerationEU and Romanian Government, under the National Recovery and Resilience Plan for Romania, contract no. 760007/30.12.2022 with the Romanian Ministry of Research, Innovation and Digitalisation through the specific research project P2-Sustainable energy & environments.

ABSTRACT In soft-core processor implementations based on software schedulers, there is an overhead that should not be neglected in HARD and FIRM real-time systems (RTS). Due to interrupt service routines (ISRs), which are asynchronous with the processor and have a higher priority than thread execution, the response time of an execution thread interrupted by an asynchronous ISR may exhibit jitter. The overall purpose of this paper is to present the hardware scheduler accelerator (HwSA) concept based on the RISC-V 4-stage pipeline CPU, where the scheduling algorithm is implemented in hardware. The proposed system is compared with existing soft-core processors using different parameters, such as the field-programmable gate arrays (FPGA) resources used for implementation, frequency, instruction set architecture (ISA), scheduler type, and the number of pipeline stages. Priority inversion is a scheduling concurrency issue in RTOS, in which a high-priority task is indirectly blocked by a task with lower priority. The scientific contribution within this project is given by a dynamic scheduling algorithm implemented in hardware, support for resolving the priority inversion situations and the use of RISC-V ISA provided under open source licenses. The analysis shows that the proposed system has higher performance in terms of the event response time and lower jitter for the scheduler, as well as higher FPGA resource utilization compared to existing designs. The support for solving priority inheritance, as a derivative contribution presented in this paper, is represented by the HwSA_RTOS hardware scheduler and dynamically changing the priorities of a task (instPi). The corresponding instPi priorities set by the mrPRIscPUi register can be changed based on hardware logic since HwSA_RTOS handles multiple prioritized events at each instPi, including the mutex event lr_enMutexi. The results obtained achieve the proposed objectives since the execution threads have their hardware context, mainly following the real-time deterministic response of the system. We evaluate FPGA resource utilization, operating frequency, and processor performance jitter. From the implementation results, we show that HwSA_RTOS achieves 10,82% (FFs) and 14,84% (LUTs) FPGA hardware resource utilisation improvement compared with nMPRA + nHSE, which is based on MIPS32 and 5-stage pipeline implementation.

INDEX TERMS Hardware RTOS, real-time systems, microprocessor, field-programmable gate arrays (FPGA), scheduling.

I. INTRODUCTION

Real-time systems are becoming mainstream in our daily lives. New developments and systems are demanding

The associate editor coordinating the review of this manuscript and approving it for publication was Shih-Wei Lin¹.

real-time capabilities every day. Many of these systems have emerged with the new era of Internet of Things (IoT) devices. These devices require not only real-time capabilities but also high-performance characteristics to develop systems with predictable task execution. The non-deterministic behavior triggered by asynchronous external events and events

generated by inter-thread synchronization and communication mechanisms is because, since for most real-time operating systems (RTOSs), real-time applications are executed in a variable number of cycles, mainly due to the software, ISA, and CPU architecture implementation. Related architectures and hardware implementations should provide feasible scheduling, even if the total CPU usage is close to the maximum limit. The limitations of current CPU implementations are determined by the design level of the processor, software, memory, I/O subsystem, high-level languages, and their compilers. All these factors lead to a variable execution time due to the RTOS behavior that implements the designer in the software—that is, an unpredictable response to RTS stimuli.

The research carried out in the field of microprocessor architectures proposes certain concepts that integrate both the functions of RTOSs and the task scheduler, these being totally or partially implemented in hardware. Thus, scheduling algorithms, as well as the synchronization and inter-task communication mechanisms, are integrated into hardware, guaranteeing a characteristic response time for the RTSs [1], [2]. In these projects, the software still has the role of switching thread contexts. Under certain conditions imposed by severe RTS time constraints, in some cases, RTOSs do not provide the performance required for real-time control, and at the same time, the kernel overhead is too high. In this context, when using hardware RTOSs or RTOS accelerators, advantages are offered, such as lower CPU resource usage and a high level of real-time performance based on fast application programming interface (API) execution and quick interrupt response. An RTS needs a predictable time management scheduler to guarantee system timing properties (such as task execution periods and deadlines) [3]. Besides these, a real-time scheduler must also ensure proper resource utilization and maximum throughput. This paper addresses the scheduling problem considering a set of real-time tasks and the Xilinx FPGA-based development kit. The proposed concept implements scheduling capabilities in hardware on CSR RISC-V attached to a single-core 4-stage RISC processor running at 33 MHz, instead of being performed in the software using a traditional RTOS. A hardware-based scheduler increases system predictability and reduces the complexity of scheduling analysis for real-time tasks [4], [5]. The feasibility of scheduling periodic tasks within RTSs with fixed priorities has always been an important research topic [6]. Thus, feasibility tests have been proposed to guarantee the timing requirements of RTSs, such as Scheduling Point Tests and Response Time Tests. In addition, the technological and architectural development of multicore processors [7], [8], creates new research opportunities for energy-efficient task scheduling [9].

The solution proposed in this work as a practical contribution is based on the multiplication of RISC-V 4-stage pipeline resources (thread switching subsystem based on multiplied hardware resources – tsSMHwR), patented in Munich, Germany, the design in hardware of a preemptive scheduler (HwSA) and the RTOS mechanisms implemented

in hardware, referred to as HwSA_RTOS (a unified acronym for HwSA, tsSMHwR and RTOS API). These solutions have the advantage of a minimal time period required to handle events associated with processor instances, leading to a predictable thread context switch time, minimum event jitter, and increased overall performance. Assigning external events to execution threads, allows the application manager to improve the CPU execution behavior. Based on the multiplication of resources in the processor datapath, the pipeline registers store information for each thread. Switching thread contexts in the proposed soft-core CPU does not require more than 1-2 clock cycles. The paper describes the system verification and analysis of a predictive scheduler for real-time applications, providing detailed discussions on implementing the tsSMHwR + HwSA concepts in FPGA. The innovative processor implementation named HwSA_RTOS represents a feasible and realistic alternative to pre-existing solutions to meet the current rigorous requirements imposed by critical RTSs. Fig. 1 shows the HwSA_RTOS (HW-RTOS) block diagram using a system-on-chip (SoC) design, including control and status registers (CSR) block.

The motivation for such an implementation is the reduced number of microarchitecture implementations that can provide both efficiency and determinism for real-time applications. The concept implemented in this paper is to move the software scheduler to a hardware component. The hardware component has several strong assumptions: each task has a hardware timer (a single hardware timer can also be used); to enable task context switching with minimal jitter, task information is stored in the register file (RF) so that context switching can take place almost immediately (1 or 2 clock cycles); existing RTOSs can be compatible with the proposed architecture. The authors present an FPGA implementation for the performance of fast task context switches. The main architectural modifications as compared to a conventional pipeline processor are as follows: 1) redundant task state and registers and 2) a scheduling module that can quickly manage threads and their associated events. System analysis consists of FPGA resource estimation using Xilinx Vivado DS, as well as a real-time timing evaluation of the external event response time using an oscilloscope.

Although the design datapath is significantly different from classic RISC-V 4-stage pipeline implementations, the hardware scheduler and the concept of multiplied resources are comparable to the approaches used in modern GPUs. The HwSA_RTOS project presented by the authors is based on the RISC-V multiplication of CPU resources patented by the authors. This paper presents an original implementation of a dedicated processor based on multiple pipeline registers and support for a dynamic scheduler implemented in HwSA based on RISC-V ISA, for real-time applications using sensors with low response times and high predictability. Following the presentation and description of the HwSA_RTOS architecture and the analysis of the processor architectures with functions implemented in hardware, as scientific and practical contributions in the field

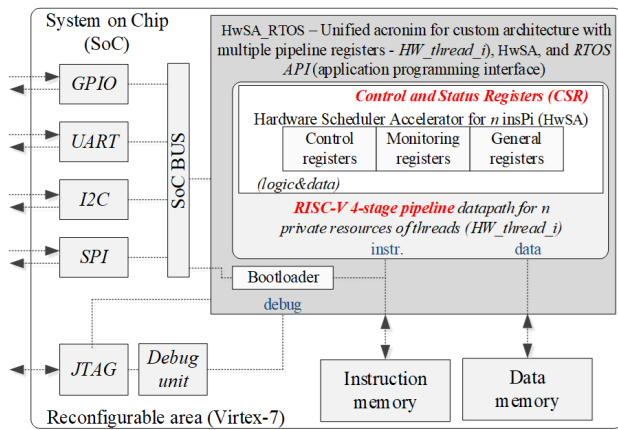


FIGURE 1. Block diagram of the SoC that includes hardware scheduler and shared RISC-V datapath.

of computer architecture, this paper presents the following achievements:

- A feasibility study on the hardware implementation based on the HwSA unit (subsection III-A), thus facilitating the practical testing of the proposed architecture using programmable logic technology;
- A modeling datapath and control unit (Fig. 2) to meet the imposed HwSA unit requirements;
- Reorganization of the HwSA unit registers, event logic, and their interconnection with the multiplexed resources (subsection III-B and III-D);
- Integration into the HwSA of the module for external events handling, mutexes, and inter-task messages, using a unified space of priorities for external events and tasks.

The rest of this paper is organized as follows. After the introduction in Section I, Section II reviews the HW-RTOS FPGA designs and soft-core CPUs. Section III presents the hardware-accelerated RTOS based on the real-time event-handling module. Section IV describes the results of the FPGA synthesis and implementation project, and section V respectively present the discussions. Finally, Section VI summarizes the conclusions of the paper, together with further research implications.

II. BACKGROUND AND RELATED WORK

RTOS is a software component that controls the execution of application programs, acting as an interface between applications and hardware. The OS hides the details of the hardware side from the programmer while providing the most API for more efficient use of the whole system. A typical OS guarantees services in the following areas: task creation and execution, I/O device access, file access control, system access, error detection and correction, and performance monitoring. The single-cycle I/O interface is a unique feature of the Cortex-M0+, which allows this processor to perform rapidly in I/O control tasks. In most other Cortex-M processors [10], bus interfaces are based on the AHB Lite or AHB5 protocol,

which is a pipeline bus protocol and allows operations at a high frequency. However, it requires two clock cycles per transfer. The single-cycle I/O interface adds a straightforward non-pipeline interface for connecting to a small set of device-specific peripherals, such as the GPIO (SGPI). When combining this feature with the penalty given by the Cortex-M0+ processor's jump instructions (because its pipeline is only two stages), many I/O control operations can be performed faster than most microcontroller architectures.

In [11], a new CPU architecture called RISC-V3 is presented, which is compatible with the RISC-V instruction set but uses the internal Redundant Number System (RNS) representation to speed up the execution time of the instructions and, therefore to increase the system performance. RISC-V is highly suitable for RNS, as it does not have a flag register, which is expensive to calculate when using an RNS. Furthermore, a simulator of instruction sets was used to estimate the performance of the proposed system for a reference suite (Embench). The processor proposed in [11], which uses RNS as the default representation, re-converts data only for specific operations, such as logical operations. The results presented in [11] show that an improvement of up to 81% has been achieved using the RISC-V3 architecture compared to a binary one, depending on the benchmark executed and the CMOS technology. Another application wherein a hardware scheduler can be successfully used is described in the paper [12], where the authors developed a multi-core processor to Advanced Encryption Standard (AES). The multi-core processor architecture is based on packages. The authors defined a 32-bit data package sent by the user to configure the internal scheduler registers of the encryption algorithm. The hardware scheduler was implemented using the Verilog HDL and Synopsys tools. For synthetization and implementation, they used an FPGA chip with 65 nm integration technology. For the practical part, the authors simulated the entire architecture at the logic gate level using the configuration file of the chip's internal circuits with FPGA. The architecture validation was carried out by sending successive messages to the processor and checking the correctness of the scheduler commands. Built on the memory latency analysis model proposed in [13], the new concept determines whether each matrix object should be accessed via on-chip Block Random Access Memory (BRAM), a CPU L2 cache shared, or directly from DDR memory. The authors present an overview of the Xilinx Zynq-7000 programmable SoC architecture and the proposed framework.

In [14], the authors describe the design and implementation of a 256-bit very-long instruction microprocessor (VLIW) based on the new RISC-V ISA. The proposed architecture packs eight 32-bit instruction flows, each performing fixed operational functions to create a 256-bit instruction format. The contributions relate to the proposal of a general-purpose VLIW architecture and the integration of a dynamic instruction scheduler to overcome the lack of a specific VLIW RISC-V compiler for the proposed VLIW architecture scheduler.

TABLE 1. FPGA resource requirement comparison among soft-core CPUs.

	A.P. FFT RISC-V [16]	PROPOSED HWSA_RTO S RISC-V	MICRO BLAZE [17]	ARM CORTEX- M3 [18]	XUM [19]	RPU [20]	NIOS [21]	PICOR V32 [22]	NEO RV32 [23]	DARK RISC V [24]	UE RISC- V [25]	NMPRA + NHSE (MIPS32) [26]
FFs	361	7765 (2664 FOR HWSA)	1627	6378	2469	1103	5231	1085	992	246	2289	8708
LUTs	1897	13752 (4476 FOR HWSA)	1913	13844	1548	2943	8238	2019	2338	1177	3676	16149

The Pfair scheduling algorithm [15] is suitable for real-time environments, but it can become resource-inefficient when implemented in serial software. In this case, software only works serially if it is executed by a single-core processor. Parallel-running software is executed by multiple processors simultaneously or by several cores in a single processor. The processing time for the Pfair algorithm increases linearly with the number of tasks because the necessary hardware logic also increases with them. This is because new blocks add uncertainty to the use of the system, and its execution time will increase with the number of tasks. If the algorithm is implemented in hardware, the scheduling process is accelerated. It works by proportionally calculating the progress for each task of its execution period and time. Experimental results demonstrated that the scheduling times were improved while the power consumed was reduced by up to 5 times. The contributions presented by the authors in [16] refer to the design and implementation of a dedicated RISC-V-based microprocessor architecture with only 20 instructions for the Fast Fourier Transform (FFT) algorithm with arbitrary points. Validated experimental results show that the presented system can complete the FFT algorithm at any time by reconfiguring the software parameters and expanding the memory capacity. It is suitable for embedded applications due to its minimal FPGA resource requirements (1897 LUTs, 361 FFs, and 25 BRAMs) and low power consumption (2.016 W at 33 MHz). Tab. 1 presents the Artix-7 FPGA resource utilization based on different soft-core implementations, including the tsSMHwR + HWSA RISC-V project (4 instPi) with the dynamic hardware scheduler. Process-based OSs use the general concept of a process, which is an instance of a program executed by one or more threads of execution. In the context of this research, threads have a separate hardware context and inter-thread communication requires minimal resources with lower overhead. In process-based multitasking, two or more processes can be run simultaneously by the OS, each program having its own address space with a high cost in terms of kernel overhead and related context switching time. Thread-based OSs produce more interrupts than process-based OSs because of shared program objects and shared data between threads of the same process. Therefore, the efficiency of the proposed method will be improved if the OS is based on the hardware implementation of RTOS functions. Thus, threads have associated dynamic priorities and separate hardware contexts, and their execution is correlated with several types of events that inherit the thread priority.

III. ASYNCHRONOUS EVENT PROCESSING BASED ON HARDWARE-ACCELERATED RTOS

A. HWSA_RTOS OVERVIEW

The 32-bit HWSA_RTOS processor complies with RISC-V ISA designed and validated using the xc7vx485tffg1761-2 FPGA circuit produced by Xilinx. The current design includes a standalone processor running at 33 MHz, as well as a complete System-on-chip (SoC) project developed, simulated, synthesized, and implemented on a targeted Xilinx FPGA Virtex-7 (XC7VX485T-2ffg1761C). The private resources of threads are referred to as HW_thread_i, with $i = 0, \dots, n - 1$. The hardware instances (instPi) of a thread execute and allocate only one HW_thread_i type resource at a given time. The terms thread and task are used interchangeably in this paper. However, the thread context (HW_thread_i) incorporates the CPU registers, instPi environment block, and kernel stack, as well as a user stack associated with the process. Regarding the HWSA_RTOS architecture, it is defined by Eq. (1).

$$\begin{aligned} \text{instPi} = & \text{BLname(PIPELINE_STAGE)} \\ & + \text{HWSA_RTOS(TYPE)} + \text{HW_thread_i} \\ & + \text{MULTIPLEXER(PIPELINE_STAGE)} \quad (1) \end{aligned}$$

Here, instPi represents instance i for tsSMHwR, and BLname (PIPELINE_STAGE) is the baseline name with the number of pipeline stages used. HWSA_RTOS (TYPE) represents the RTOS implemented in hardware for n threads, and TYPE can be implemented as a hardware block that uses control and status register (CSR), standalone, etc. HW_thread_i constitutes the private hardware resources for thread i, and MULTIPLEXER implements the combinational resources shared by threads, which is dependent mainly on the number of pipeline stages. For RISC-V and ARM ISA, Eq. (2) and Eq. (3) can be written.

$$\begin{aligned} \text{instPi} = & \text{BL_RISCV32IM_uRV}(4 \text{ pipeline stages}) \\ & + \text{HW_thread_i} + \text{HWSA_RTOS(CSR)} \\ & + \text{MUX}(4 \text{ pipeline stages}) \quad (2) \end{aligned}$$

$$\begin{aligned} \text{instPi} = & \text{BL_ARM_CM3_MCU}(3 \text{ pipeline stages}) \\ & + \text{HW_thread_i} + \text{HWSA_RTOS(sys_ctrl)} \\ & + \text{MUX}(3 \text{ pipeline stages}) \quad (3) \end{aligned}$$

In the following, the HWSA_RTOS architecture characteristics will be presented:

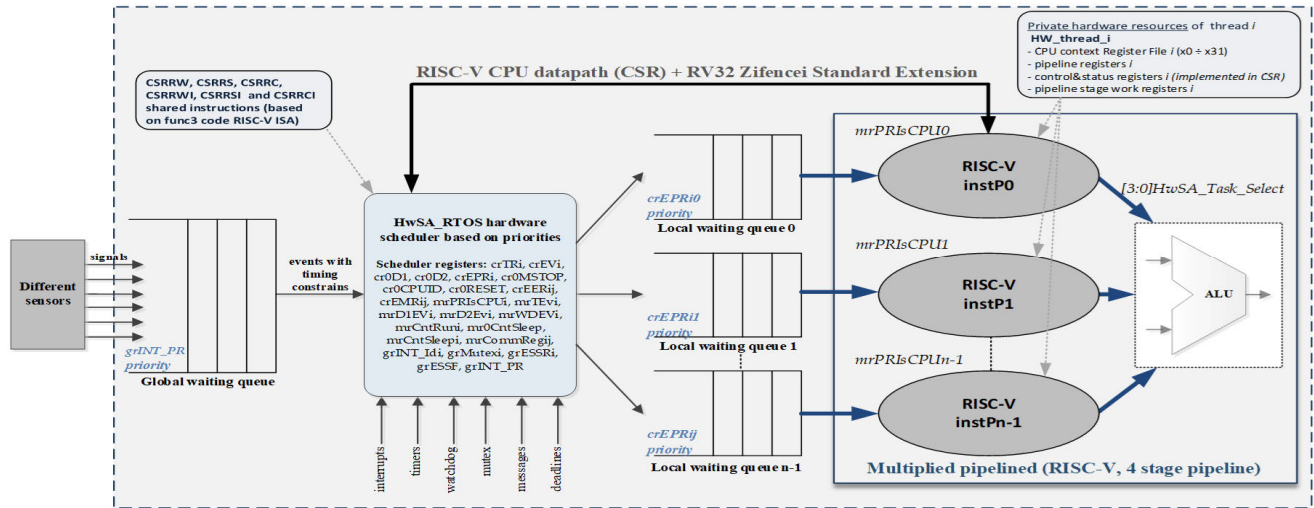


FIGURE 2. HwSA_RTOS + tsSMHwR architecture based on hardware event handling module (HwSA).

- Structure of the Reduced Instruction Set Computer (RISC) pipeline stages, similar to the RISC-V architecture;
- Four-stage multi-pipeline architecture with hazard detection and forwarding unit;
- Scheduling of a total of 16 tasks (instPi) and external events (ExtIntEv[p]) for small real-time applications;
- Tasks and external events with fixed priorities (mrPRI-instPi);
- Preemptive execution based on HW_RTOS scheduling;
- Centralized offline scheduling for instPi and prioritized events (crEPRI);
- Tasks and external events activated in hardware through specially implemented functions;
- Switching task contexts with zero theoretical overhead because the stack concept is not used to save and restore task contexts (HW_thread_i).

To implement HW_thread_i registers in FPGA, the required memory is directly proportional to the number of instPi (Fig. 2). To ensure predictable task execution, a fixed scheduling scheme was chosen based on the priority address encoder developed in the scheduler. The hardware implementation of the desired scheduling algorithm remains at the designer's choice, depending on the frequency of task execution or the application type controlled by RTOS. Therefore, the HW_thread_i remapping operation and timer blocks developed in HwSA are valuable mechanisms that allocate space to implement a predictable round-robin scheduling scheme. Regarding the spatial isolation of the tasks, a special RF implementation was chosen that, in addition to remapping the contexts of the tasks, has the advantage of introducing a minimum predetermined jitter.

B. HwSA_RTOS HARDWARE IMPLEMENTATION

The implementation complies with the organizational structure of the RISC-V 4-stage pipeline architecture, introducing some special instructions for controlling the hardware

scheduler. This incorporates the scheduler utility into a separate functional block that is part of the processor, eliminating the specific disadvantages of current software schedulers. In the tsSMHwR architecture, external events are handled as tasks, with the scheduler implementing a dynamic interrupt handling mechanism. Fig. 2 validates both the HwSA_Task_Select command signals and CPU datapath signals. Switching of task contexts is controllable by the scheduler using the HwSA_Task_Select selector.

The novelty brought by the authors through this research paper consists in reducing the time needed for the scheduling operation and partially eliminating the overhead given by the prioritization of multiple events attached to the same instPi with the identifier instPi_ID. Each instPi can have multiple events attached validated in the crTRI register, such as time events (lr_enTi, lr_enWDi, lr_enD1i, lr_enD2i), external events (lr_enInti), mutex events (lr_enMutexi), and message sync events (lr_enSMi) defined in [26]. The crEvi register implemented in CSR indicates the occurrence (1 L) or not (0 L) of a corresponding event validated in crTRI.

Within the proposed CPU datapath, each task has a program counter (PC), an RF, and a set of pipeline registers. The HwSA_RTOS concept uses a common space of priorities for external events and tasks, so low-priority events cannot affect HARD real-time tasks in the system. The handling of asynchronous external events, whereby they can be attached to tasks, involves situations where external events are treated as tasks, thus inheriting the priority of the task to which they were attached. For this, the HwSA unit comprises the logic of selecting active events, which has the role of arbitrating and scheduling each instPi and its associated events. This can be performed directly, usually through the enabled scheduler (either statically or dynamically).

The execution of a scheduled task is achieved by simply remapping the multiplexed hardware resources (HW_thread_i) related to instPi that are to be executed due to a higher-priority event. In hardware and Verilog HDL, all

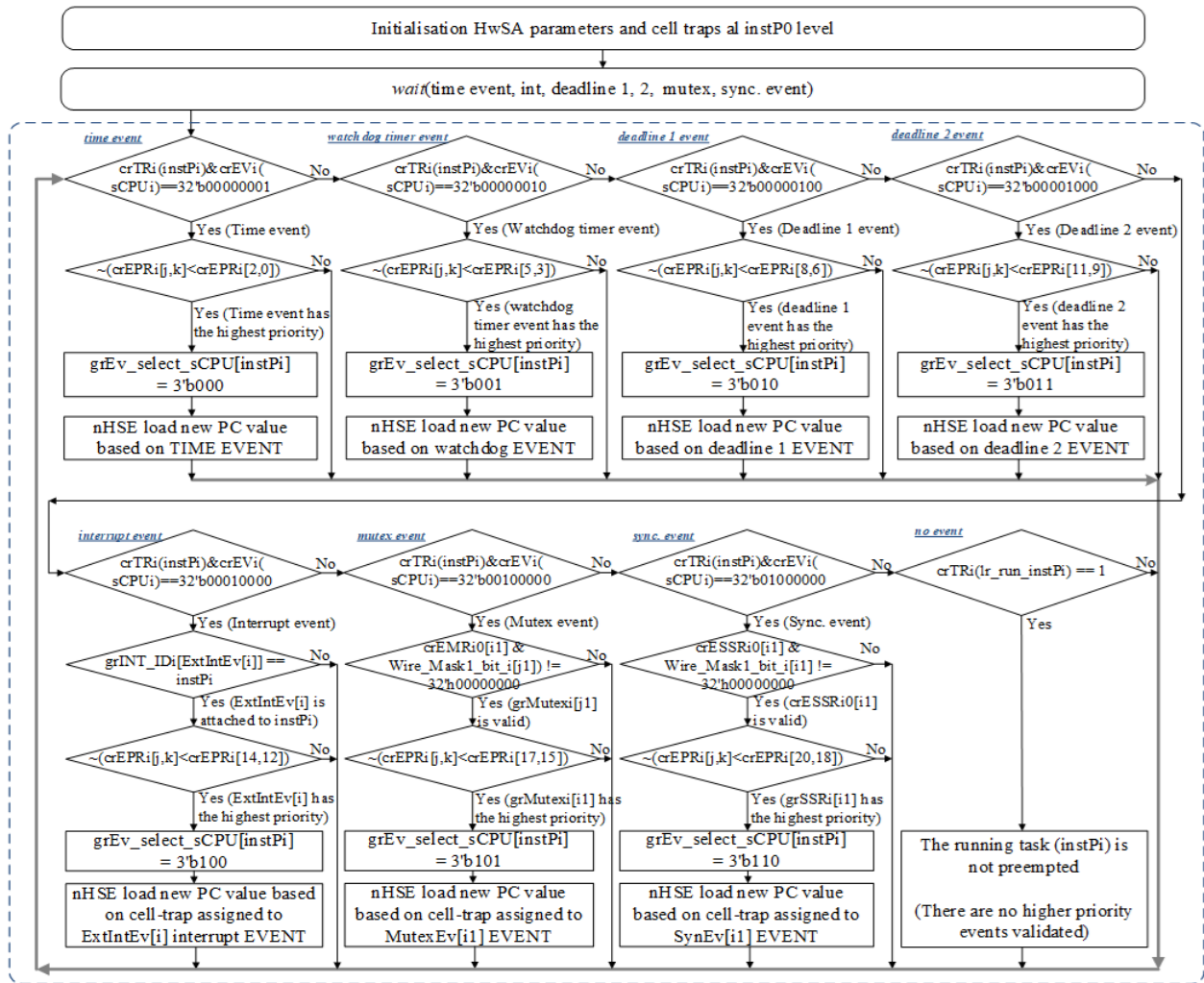


FIGURE 3. The hardware solution for handling events is implemented in the HWSA scheduler unit.

logical structures are implemented separately, and signals are propagated in parallel. Thus, the system clock must control only the modules corresponding to this processor cycle simultaneously.

To demonstrate processor functionality, traditional RISC-V compilers can easily be used to create real-time applications. The static scheduler performs preemptive scheduling according to $instP_i$ priorities (task 0 running on $instP_0$ has the highest priority, and task $n - 1$ running on $instP_{n-1}$ has the lowest priority). In this case, the priorities cannot be changed in any way. The static scheduler is activated at CPU reset and can be activated or disabled only by task 0 (executed by $instP_0$). A dynamic scheduler has support for dynamic scheduling, allowing one to change the priority of any task (implicitly of an $instP_i$). It can be enabled or disabled only by $instP_0$, being enabled at reset. The proposed hardware dynamic scheduler can also be used to resolve issues related to priority inversion or change the priority depending on the device status, among other functions. Asynchronous events handled by the HWSA are represented by the following signals: time events (lr_TEvi), events gen-

erated by the watchdog timer (lr_WDEvi), two events for deadlines (lr_D1Evi , lr_D2Evi), external events that can be attached to a task i (lr_IntEvi), mutexes used to access shared resources ($lr_MutexEvi$) and events for synchronization and communication between tasks (lr_SynEvi). The scheduler initialization is performed upon reset in the `HWSA.v` file and the code corresponding to the attachment of external events and their related trap cells are located in the `Parameters.v` file. Four counters are implemented in the `TIMERS` module, namely `TCounter`, `WDCounter`, `D1Counter`, and `D2Counter`, for each $instP_i$. When the counter exceeds the preset value in the dedicated registers ($mrTEVi$, $mrWDEVi$, $mrD1EVi$, $mrD2EVi$ defined in [26]), the event bit assigned to this counter is set (lr_TEv , lr_WDEv , lr_D1Ev , lr_D2Ev), if the last four bits of the $crTRi$ register that activate the four events are also set (Fig. 3).

The `MUTEX` module contains the `Mutex Register File (MRF)` registers dedicated to mutexes ($grMutexi$). The MRF contains the state of the mutex and a field for the owner's task identifier. These registers are global so they can be accessed by any $instP_i$.

Because synchronization and inter-task communication are achieved through shared memory, changing instPi contexts must ensure the consistency of data used for these mechanisms, eliminating serious situations of data corruption when the scheduler executes a preemptive algorithm. In the current implementation of this project, the task context remapping is triggered at the rising front of the HwSA scheduler clock, a signal that coincides with the CPU clock. Attaching these events to instPi causes them to borrow the priority of the task to which it is assigned, and there is a unified priority space for external events and tasks. Also at the level of each instPi, a set of events attached to it is prioritized separately via the crEPRI register, with 3 priority bits for each event in the same category. The dynamic scheduler allows changing the mrPRIscPUi priorities of the processor instances. In the case of priority inversion situations, when two tasks share an exclusive resource, the scheduler can raise the priority of the instPi task (mrPRIscPUi) associated with that mutex (Mutexi), determining by hardware the highest priority associated with a thread requesting the mutex. In the algorithm implemented in HwSA, it is checked if the Mutexi bit in the EMR is set and if this event is validated by the lr_en_Mii entry in the crTRI register. Thus, the scheduler logic selects the corresponding instPi once this mutex has been released by another task. Searching for mutex events by hardware logic, prioritizing them by crEPRI, and fast context switching of tasks based on HW_thread_i reduces response times and gives applications a predictable response with minimal overhead.

C. REAL-TIME SCHEDULER IMPLEMENTATION LOGIC

Most of the real-time task scheduling algorithms used in embedded RTSs are based on worst-case execution time (WCET) analysis to meet specific deadlines. Generally, tasks (τ_i) executed on embedded real-time multitasking systems are periodic, and each τ_i is described by seven parameters (Eq. (4)).

$$\tau_i = (S_i, WCET_i, BCET_i, AET_i, D_i, T_i, P_i) \quad (4)$$

Here, S_i is the time point at which a real-time task becomes ready or is activated to run; $BCET_i$ is the best execution time; $WCET_i$ is the worst-case execution time; AET_i is the actual execution time, which varies during each request; D_i is the relative term of the τ_i load; P_i is the relative priority of τ_i task; T_i is the period of τ_i task. If the k -th ($k \geq 1$) scheduling request for the execution of the periodic task τ_i reaches S_i , the k -th execution in response to its request begins at a given time $S_i + (k - 1)T_i$, and its absolute term must be calculated as follows: $S_i + (k - 1)T_i + D_i$. The execution of a periodic task is called an instance or execution request, and $BCET_i \leq AET_i \leq WCET_i \leq D_i$, $D_i = T_i$.

The deadline miss percentage is defined as the number of deadline misses divided by the total number of ready task instances of an embedded real-time software application system in a given sampling period [27]. Static algorithms assign priorities during design. All assigned priorities remain

constant throughout the creation of a specific task. Dynamic algorithms assign priorities during execution, based on task runtime parameters. Dynamic programming can implement either static or dynamic priorities. Rate Monolithic (RM) and Deadline Monolithic (DM) are practical examples of dynamic programming based on static priority. Earliest Deadline First (EDF) and Least Slack Time First (LST) are dynamic scheduling algorithms with dynamic priority. EDF and LST are optimal for RSTs only if the scheduled tasks are preventive, there is only one processor, and they are not overloaded. However, the main limitation of these algorithms is that their performance decreases exponentially if the system becomes slightly overloaded [28]. For periodic tasks, the system load, denoted by L , can be determined using Eq. (5). Here, m represents the scheduled task number, E is the task execution time, P is the task period, and D is the task deadline. Therefore, $Q = P$ if $P \geq D$, and, at the same time, $Q = D$ if the true condition is $P < D$.

$$L = \sum_{i=1}^m \frac{E_i}{Q_i} \quad (5)$$

The system is overloaded when a preemptive scheduler cannot feasibly schedule RTS tasks within the imposed WCET. A reasonable method to measure the performance of the scheduling algorithm during a system overload is to measure the scheduler jitter according to the implemented algorithm. Performance metrics are based on CPU efficiency utilization (ECU), providing information on how efficiently the CPU is being used, as defined in Eq. (6).

$$ECU = \sum_{i \in R} \frac{V_i}{T} \quad (6)$$

Here, V_i is the value of task i , R is a set of tasks executed by the CPU, and T is the total scheduling time. The task value is equal to the execution time if and only if the task execution is completed within its deadline. If the V_i is equal to zero, the task does not meet the deadline. Therefore, the most appropriate measure of performance is the success rate (SR), which is defined in Eq. (7).

$$SR = \frac{\text{Number of tasks successfully scheduled}}{\text{Total number of tasks arrived}} \quad (7)$$

The execution time required for each scheduling algorithm is critical, especially when working with HARD and FIRM RTSs. The results of the EDF, RM, and DM algorithms in the same environment, measured in terms of SR and ECU, determine the scheduling feasibility of a set of tasks. The execution time taken by each scheduling algorithm is also a defining element of WCET [29]. The logic for hardware events handled by the HwSA unit is illustrated in Fig. 3. Additionally, the hardware support in implementing the synchronization mechanism is represented by the MRF for mutexes. It contains several special registers called grMutexi, which memorize the mutexes' status and the instPi owner identifier. The number of mutexes and grMutexi registers depends on the HwSA implementation.

For the current HwSA_RTOS implementation, this number has been denoted by m , and MRF is constituted based on m and $instPi$. The $grMutex_i$ register selects the $instPi$ identifier to which mutex i was attached. Since resource sharing is carried out between all $instPi$ in $tsSMHwR$, the $grMutex_i$ global register owners can be accessed by any $instPi$ under the direct control of the HwSA unit. Regarding the real-time aspect, it is essential to avoid task execution preemption when a mutex change occurs, the operations with these entities being atomic. As soon as the expected mutex is unlocked, the suspended task is reactivated by the HwSA (Fig. 3), entered in the READY state, and executed according to its priority ($mrPRI_{CPUi}$).

To choose the most suitable solution for synchronization and inter-task communication mechanism implementation, the HwSA_RTOS API implements, at the level of the HwSA block, a set of $grMutex_i$ registers with fast access. Each $grMutex_i$ register has a bit of memorizing the mutex status and an $m - 1$ bit to memorize the proprietary $instPi$ identifier. When a task i running on the $instPi$ task hardware instance receives a message or is waiting for a mutex to be released, it is necessary to identify the source of this event. Since the $grMutex_i$ registers are accessed by all $instPi$, blocking and releasing a mutex must be carried out individually. When the $Mutex_i$ bit has a value of 0 L, the $grMutex_i$ mutex is considered to be available, so that an $instPi$ can take only one mutex at a time. Since the implementation of these mechanisms is performed in hardware, the process required to search for the message source or select the $instPi$ waiting for a mutex can be completed in an RTS characteristic time.

The $crEMR_{ij}$ control register represents the mutex validation register implemented by the HwSA. After the CPU reset operation, all bits are at 0 L, and i is implementation-chosen. Thus, an M_j bit ($j = 0..31$) represents an enabled (1 L) or an inhibited (0 L) mutex. If $M_j = 0$, the mutex cannot produce a $MutexEvi$ event. The global registers $grMutex_i[3:0]$ contain the IDs of the $instPi$ that are associated with the mutexes, which can be accessed by any $instPi$. The Signal Synchronization and Message Register File (SSMRF) defines the synchronization type events; after reset, all bits are set to 0 L.

The $grSSMR_i$ global registers are used to implement the synchronization events; the value 0 L denotes a message-type communication event inhibited, and 1 L denotes a validated

event. The instructions for working with the $grSSMR_i$ registers and the hardware block dedicated to the implementation of this mechanism are designed at the level of the HwSA block, generating the signals corresponding to the modification of the registers in the SSMRF. Thus, when one or more tasks are waiting for an SM-type event, the HwSA preemptive scheduler checks the $grSSMR_i$ registers in one clock cycle based on a Content Addressable Memory (CAM) search.

D. HwSA VERILOG HDL SOFTWARE IMPLEMENTATION

The design of this scheduler has been divided into several stages. Each of these was completed before the next stage, post-synthesis simulation being a crucial stage in FPGA design. The setting and reset operations on the $Mutex_i$ bit are performed using the scheme in Fig. 4.a, indivisibly, in a single processor cycle. The algorithm for setting the bits in the $crEMR_i$ register is shown in Listing 1.

According to Fig. 4.b, a test, and set instruction (signals in_tasm_wr and $Address_i$) will read the old value of the corresponding mutex bit from the MRF ($Mutex_i$) and set the $Mutex_i$ bit. At the level of each $instPi$, there is a scheme similar to the one illustrated in Fig. 4.c that allows the generation of a $MutexEvi$ -type event whenever an expected mutex is free. Based on $instPi$ hardware instances, it can be decided which mutex is taken into account with the help of signals $lr_en_M_0, \dots, lr_en_M_{m-1}$. These signals are stored in local registers called Enable Mutex Registers (EMRs). There can be one or more EMR_i registers depending on the number of mutexes implemented in the MRF. For synchronization with the CPU clock, the type D flip-flop is used, which takes the information to the rising front of the processor clock. If the $Mutex_i$ bit was on 1 L, it was acquired, and the last n bits of the MRF register (the $Address_i$ signal) represent the mutex owner ID, which is provided by the in_tasm_wr , $Address_i$, and $/Mutex_i$ signals. If the mutex bit is on 0 L, it will set it to 1 L and write the $instPi$ ID, from which the instruction was executed, in the corresponding MRF register (Fig. 4.b, for a bit of the MRF other than the $Mutex_i$).

If the $Mutex_i$ bit was on 0 L, then the mutex is free, and the $instPi$ ID is of the mutex owner. The mutex release instructions set the in_clrm_wr and $Address_i$ signals (Fig. 4.a) and clear the $Mutex_i$ bit. Writing to the MRF register can be performed only if $Mutex_i = 1$ and the $instPi$ ID coincides with that of $instPi$ trying to write or if $Mutex_i = 0$. For the

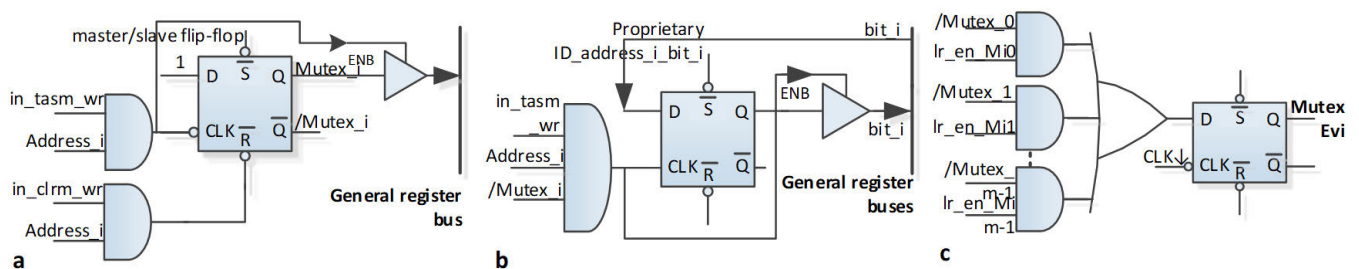


FIGURE 4. MRF logic implementation schemes (in_clrm_wr – clear mutex instruction write operation, in_tasm_wr – test and set instruction write operation).

Listing 1 HwSA_RTOS Logic of Working With Mutexes

```

1:  Set lr_en_Mii from EMRm
2:  EXIT = 0
3:  until (EXIT = 0)
4:  read the appropriate register in the MRFi
5:  if lr_en_Mii = 1 & Mutexi = 1 begin //(from EMRm) it is already taken and Mutexi = 1
6:  then
7:    instPi = IDLE //instPi wait for the appearance of the MutexEvi event
8:    lrEnMutexi = 1
9:    EXIT = 0
10: else
11:   Test:
12:   //Try writing the corresponding MRFi register with Mutexi = 1 and the instPi ID that is trying to write
13:   if Mutexi = 1
14:   //Writing fails (logical eq. must be tested when writing) (Mutexi \time (IDS + IDD)) + /Mutexi = 1,
15:   //IDS is ID-ul static, and IDD ID dynamic)
16:   EXIT = 0 // it remains in the loop
17: else // Mutexi = 0
18:   Read MRFi
19:   if MRFi matches Mutexi = 1 //mutex locked and the ID of the task that reads it, it follows that the Mutex is purchased.
20:   EXIT = 1
21: else
22:   EXIT = 0
23: if Mutexi = 0 & lr_enMutexi = 0 begin //it is possible for another instPi to take control at this point and make Mutexi = 1
24: GO TO Test
25: if Mutexi = 1 and lr_en_Mii = 0 //if Mutexi is already taken and instPi do not expect this event
26:   EXIT = 1//it comes out of the loop
27: end until

```

mutex write operation, the following logical equation must be satisfied: $(\text{Mutex}_i \times (\text{IDS} + \text{IDD})) + /\text{Mutex}_i = 1$). In the grMutexi register used for writing, the Mutex_i bit must be 1 L, and the ID must be that of the writing task. This value can be taken directly from the pipeline register so as not to cause erroneous writing. The CPU implementation steps performed for the design and testing of the tsSMHwR processor and HwSA preemptive scheduler are described below:

- Selecting the hardware design support and hardware description language (HDL);
- Creating Verilog HDL modules that represent the soft-core CPU components of the proposed concept following RISC-V RV32I ISA;
- Testing the functionality of each hardware module;
- Linking all implemented and tested modules, obtaining the CPU pipeline datapath;
- Testing the RISC soft-core processor by executing appropriate test programs, thus checking the correct instructions' execution even in the presence of hazard-type situations, interrupts, or exceptions;
- Adding the necessary support for working with peripherals (LED, DIP switches, LCD, UART), as well as testing the Bootloader module;
- Designing the logic for the CPU scheduling algorithm, MRF (listing 1), SSMRF, and HwSA-related events;

- Creating a program that demonstrates the tsSMHwR + HwSA RISC-V system's functionality and performance validation in practical real-time applications, thus verifying the predictability introduced by the hardware scheduler.

IV. EXPERIMENTAL RESULTS

A. HwSA_RTOS REAL-TIME EVENT HANDLING VALIDATION IN FPGA

The external event handler was validated via the lr_enLi bit of the corresponding crTRi register considering that instP0 has the highest priority. Fig. 5.a illustrates the time period from the asynchronous external event occurrence, corresponding to pressing a button on the Virtex-7 development kit (ExtIn-tEv[0] signal), to capture it in the crEVi register (evLi bit). The jitter is 13 ns, which depends on the time at which the trigger appears on the positive edge of the input signal (cursor 1) at the next positive edge of the CPU_clock, using the Analog Discovery 2 Oscilloscope by Digilent. The time period shown in the oscilloscope capture (DX: 302 ns) illustrated in Fig. 5.b includes the external signal triggering, setting the flip-flop in instPi_ready, setting the signal en_pipe_instP0 for instP0, jumping to the appropriate handler, executing the load word instruction and changing the digital output mapped in the data memory address space. The kernel latency, in this case, can be $1 \div 2$ machine cycles. Fig. 5.c presents the kernel latency

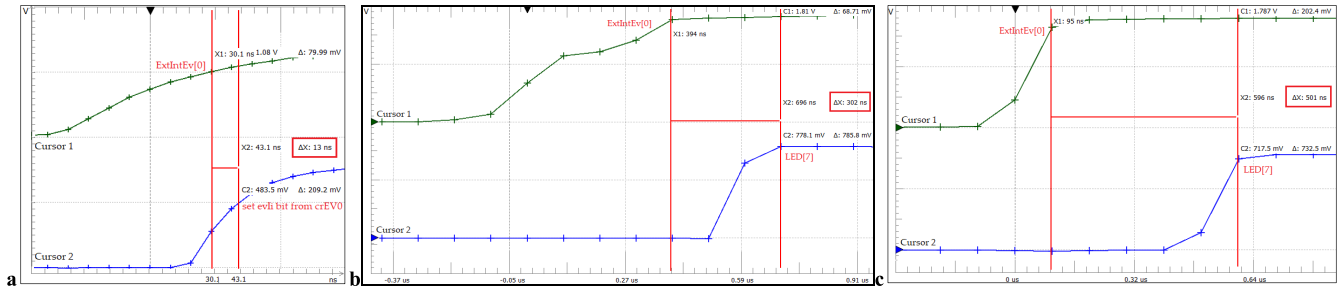


FIGURE 5. Hardware management of asynchronous external events (a) ExtIntEv[0] signal and setting the flip-flop evli in crEVi (13 ns); (b) External event handling validation and setting FPGA digital output; (c) Inter-thread communication jitter.

for inter-thread communication mechanism implementation (DX: 501 ns).

The time period illustrated in the oscilloscope capture includes the management of an external event associated with instP2 (2.18 clock cycles), message sending to the instP0 (5 cycles are required to execute the store word instruction, the parallel search in hardware, the evSMi flip-flop setting in Fig. 5.a (cycle 1), setting the instP0_ready line, changing the context based on the selection of hardware resources related to instP0 (cycle 1), jumping to the corresponding event handler (1 cycle) and executing the lw instruction to set a digital LED output 6 (4 cycles). Inter-task communication using memory areas and instP0 is based on the grSSMRi register. The variable is chosen according to the application and represents the number of registers for HW_thread_i. For the inter-task communication mechanism implementation, the HwSA_RTOS uses several grSSMF global registers, each having $2 \times n + k + 1$ bits. These registers compose the synchronization and message register file, each register using one bit to memorize the event status, $2 \times n$ bits to memorize the IDs of the source and destination tasks, and k bits to memorize the message. Fig. 6 validates the HwSA_RTOS preemptive scheduling and time event logic presented in Fig. 3, using the PicoScope 6404B oscilloscope by Pico Technology (St Neots, UK). Fig. 6.a shows the preemption points for instP0, instP1, and instP3 thread hardware instances. At time moment T1, instP0 interrupts the lower-priority task instP3, the context switch performed in one clock cycle based

on HW_thread_0. At time points T2 and T4, contexts are restored for instP3, and at T3, task instP3 is interrupted again, this time by instP1. During the monitored execution period, ending at T5, instP3 has a WCET of 4,196 μ s. Fig. 6.b shows the preemptive instP0 scheduling for a periodic time event (lr_TEvi), with instP2 and instP3 being lower-priority tasks. The time moments T1, T2, and T3 indicate the start execution moment of instP0 with the minimal jitter characteristic of RTS. The results validate the functionality and performance of the hardware preemptive dynamic scheduler, considering different priority inversion situations, thread communication, and mutual exclusion through hardware, timers, and events with call-backs and multiple configurations (n, i, m, e).

B. FPGA RESOURCE USAGE, TIMING, AND POST-SYNTHESIS RESULTS

To handle multiple events with its trap cell (external events, time, mutexes, and message-type events), a solution was proposed to integrate the priority encoder for external events and local events associated with instPi. Thus, it was possible to obtain an automatic jump to the handler for the highest-priority event associated with instPi.

The identifier register block contains one identifier for each instPi, a register with the priority attached to each instPi (mrPRIiCPUi) used only by the dynamic scheduler (except for instP0, which is always of the highest priority), and a global register containing the active instPi identifier. The following FPGA resource, according to the

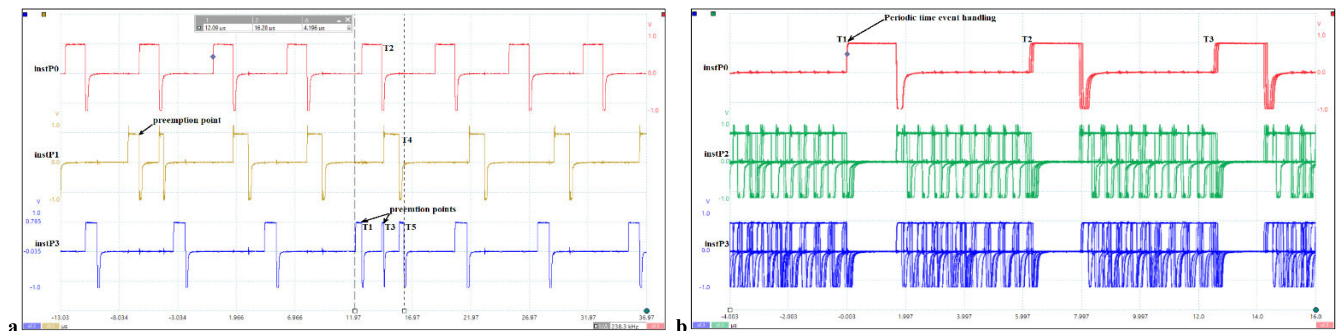
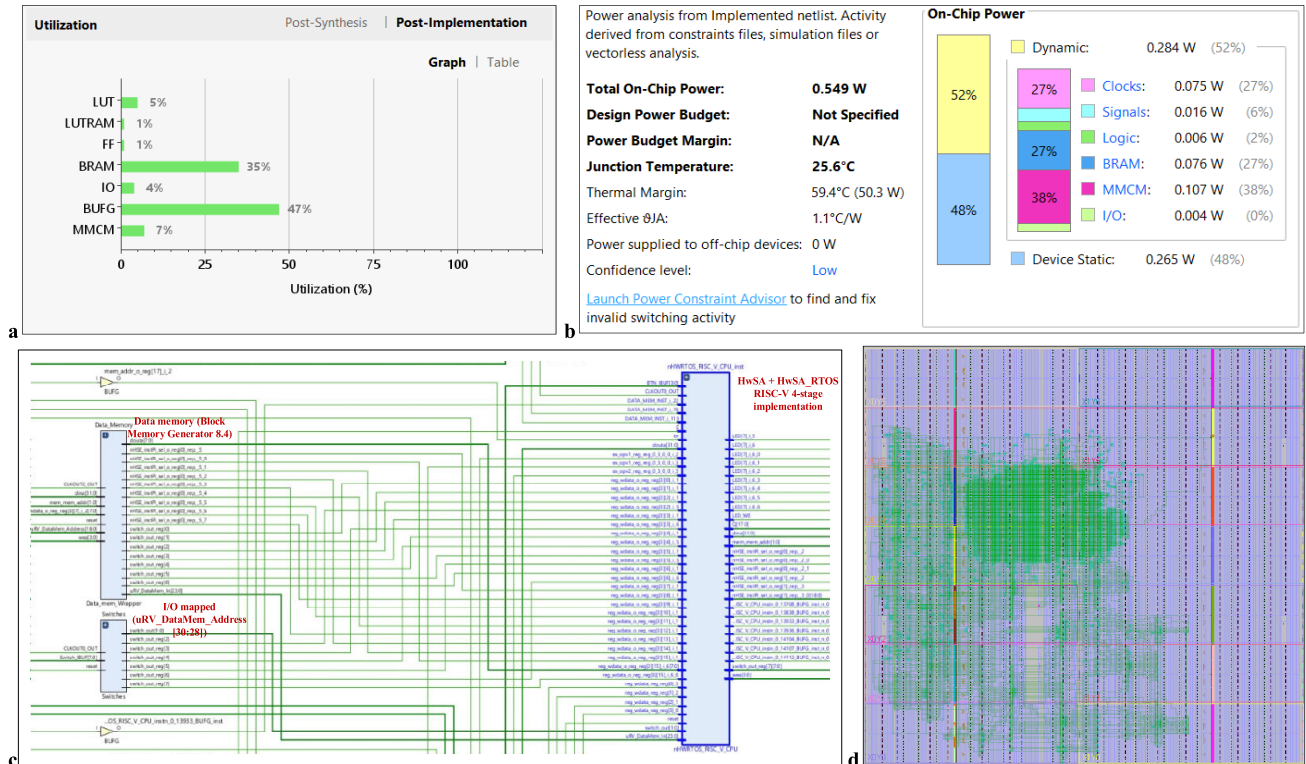


FIGURE 6. HwSA_RTOS multiple event scheduling validations (WCET instP3 = 4.196us); (a) Preemptive scheduling based on instPi priorities; (b) instP0 execution period based on periodic time event handling (oscilloscope persistent mode).



tsSMHwR multiplication degree and scheduler support, was required to implement these registers, as well as the combinational logic associated with the scheduler. In the four instPi configurations, 8476 flip-flops (FFs) and 13935 look-up tables (LUTs) were used, and for 16 instPi, 33121 FFs and 57559 LUTs, respectively, were routed into the FPGA. Thus, the post-implementation results obtained in Vivado DS concerning HwSA_RTOS's performance are satisfactory. With independent execution, the hardware scheduler has inputs for events, such as external events and timers.

Fig. 7.a present the RISC-V 4-stage pipeline FPGA resource usage using Verilog HDL, Fig. 7.b illustrate the power consumption for tsSMHwR RISC-V implementation in the FPGA (post-implementation report) obtained from the Vivado environment, Fig. 7.c and Fig. 7.d shows the nHwRTOS_RISC_V_CPU RTL schematic after synthesis and FPGA mapping of the SoC design containing the integrated hardware scheduler.

V. DISCUSSION

The widest variety of control applications lies between these two extremes. While the actions executed by these systems usually have strictly specified execution times, the processing of the data that trigger these actions can be completed promptly, only if processing latency is guaranteed and a certain upper limit is never exceeded. In this context, the information in Tab. 2 presents different processor architectures, frequencies, ISA, FPGA platforms, pipeline stages,

datapaths, and CPU characteristics. The response time in the case of data transfer through the program is within $D_x = 1 \div 32.59$ ms, depending on the location of the input line test instruction. This jitter is much higher compared to HwSA_RTOS.

Event handling logic implemented in the HwSA unit has the advantage of jumping directly to the trap cell associated with that event, eliminating software searches, and directly executing useful code. Thus, the contributions are both theoretical, when proposing the event prioritization scheme in Fig. 3, and practical by validating multiple event scheduling in HwSA_RTOS, verifying the actual preemption points with the priority-based execution. Another specific derivative contribution is to perform a comparison with a wide range of processors in terms of FPGA resource usage, datapath resource multiplication, and processor implementation architecture. A hard RTS guarantees by design that no deadline is missed in embedded applications based on RISC core architectures [30], [31]. In a distributed real-time environment, there is often an additional requirement to have different RTSs synchronized. Typically, the parameter that has the most significant impact on the architecture of the control system is WCET, namely the processing latency [32] and the microkernel. Control systems with an accuracy rate of less than one microsecond are the exclusive domain of FPGAs, Application-Specific Integrated Circuit (ASIC) for HRHS design implementation with 45-nanometer technology [33] and dedicated analog circuits. On the contrary, slow control

systems are usually implemented on programmable logic controllers (PLC) or, in many cases, on industrial computers running operating systems that are not real-time. Many RTS require high-resolution time-tick in order to operate correctly [34], and dynamically varying the scheduling functionality is commonly coupled with overhead in which some timings may be delayed, this is because RTOSs that tolerate high-resolution time-ticking potentially introduce a very large overhead on the system. Typically, a real-time kernel performs the following activities: process management, interrupt management, and process synchronization.

Process management is the main service that the OS must provide. It involves functions such as process creation and termination, scheduling, interrupt handling, context switching, and other related activities. Another kernel functionality is to provide basic mechanisms for process synchronization and communication. In classical OSs, these mechanisms are represented by semaphores that provide mutual task exclusion for shared resources. However, semaphores can lead to priority inversion situations, resulting in task execution deadlocks. As a consequence, in order to guarantee predictability, RTOS must provide special semaphores that support shared resource access protocols such as Priority Inheritance, Priority Ceiling, or Stack Resource Policy. The OS decides when an I/O device can be used for a task while controlling access to user files. The processor itself is a resource, and the OS must determine how much processor time may be allocated to the execution of a particular user program. In the case of multi-processor systems, this decision must cover the execution of all processors. The implementation presented in [18] describes a Cortex-M3 core mapped in FPGA. Furthermore, the Cortex-M4 processor contains all the features of the Cortex-M3 processor, has additional instructions to support digital signal processing (DSP) applications, and has the option to include a floating-point unit (FPU). It has the same system and debugging-level features as the Cortex-M3 processor. Lower consumption is also determined by the simpler architecture relative to microprocessors (missing or

lower-capacity cache, lower pipeline, no dynamic scheduling with speculation, etc.).

The result of this research shows that the custom CPU implementation is characterized by minimal overhead due to the scheduling operation, very low task switching times ($1 \div 2$ machine cycles), fast response to asynchronous external events, a distributed external events system with flexible assignment of external events to tasks (instPi), while guaranteeing predictable response time and easy implementation of EDF or RM scheduling. Thus, the hardware solution for handling events implemented in the HwSA preemptive scheduler is a predictable solution for handling events attached to instPi. In the proposed architecture, each memory element in the datapath was multiplied n times, resulting in HW_thread_i, i from 0 to $n - 1$. The control unit and the combinational logic circuits such as the condition test unit, the hazard detection unit, or the arithmetic and logic unit were not multiplied, they were connected to all HW_thread_i CPU resources. Each datapath hardware instance, called instPi, may at some point be in the execution state, under the control of the HwSA hardware scheduler. Thus, context switching between tasks running on instPi, with priority $mrPRI_{CPUi}$, can be performed in one clock cycle, based on the principle of HW_thread_i multiplexing in the hardware based on FPGA circuit implementation. The improvement solutions resulting from the research project will lead to tsSMHwR processors that can significantly influence the field of real-time operating systems. The contributions emerging from the current research are as follows: tsSMHwR + HwSA architecture design using Verilog hardware description language, Vivado environment and Virtex-7 FPGA development kit; Verilog implementation of multiplexed resources (program counter registers, register file, pipeline registers); Implementation and testing of the integrated hardware scheduler for task execution management and trap cell selection for external events and other events handled by HwSA unit; Implementation and testing of dedicated HwSA scheduler instructions; Hardware implementation of inter-task synchro-

TABLE 2. Comparison between different CPU architectures based on FPGA implementation and hardware support.

CPU PROJECT	CORE FREQUENCY	DEVELOPMENT KIT/ FPGA/ HDL	ISA	PIPELINE STAGES/ DATAPATH AND CPU CHARACTERISTICS
EIGHT-ISSUE VLIW CORE [14]	97.943 MHz	VIRTEX-6/ XC6VLX240T	RV32IMFD	6/ VLIW
RPU [20]	200 MHz	ARTY S7-50	RV32IMZCSR	NON-PIPELINED
NEORV32 [23]	128 MHz	INTEL CYCLONE-IV/ EP4CE22F17C6N	RV32IMCU+ZICSR, ZIFENCI	2/ 3.54 CPI
DARKRISCV [24]	100 MHz	XILINX SPARTAN-6/ XC6SLX9	RV32I	2, 3, AND PSEUDO 4-STAGE PIPELINE
ULTRA EMBEDDED RISC-V [25]	124 MHz	VERILOG 2001, VERILATOR AND FPGA FRIENDLY	RV32IMZICSR	5 (CONFIGURABLE)
PROPOSED HwSA_RTOS RISC-V (INSTP4)	33 MHz	XILINX VIRTEX-7/ NEXYS4 DDR/ XC7A100TCSG324	RV32I	4/ HARDWARE SCHEDULER ACCELERATOR
BiRISC-V [35]	87 MHz	VERILOG 2001, VERILATOR AND FPGA FRIENDLY	RV32IMZICSR	6, 7/ SUPERSCALAR (DUAL-ISSUE)
uRV RISC-V [36]	100 MHz	XILINX SPARTAN-6/ VERILOG	RV32IM	4/ 2.14 COREMARKS/MHZ

nization and communication mechanisms; Integration of the priority encoder for events implemented in hardware; Writing of machine code programs for testing the change of instPi contexts, datapath verification and HwSA scheduling unit testing; FPGA resource requirements for the tsSMHwR architecture implementation and the whole SoC project using the Virtex-7 development kit; Memory requirements analysis for HwSA_RTOS RISC-V processor implementation with instP4, instP8 and instP16; Performance evaluation for a diverse range of HwSA_RTOS 4-stage RISC-V processor configurations and implicitly HwSA (number of mutexes, number of message events, etc.) in FPGA; Perform a comparative analysis with other processor architectures in the literature.

VI. CONCLUSION

Since in this research project thread context switching is performed in $1 \div 2$ clock cycles, delays, and jitter due to multiple events characteristic of thread-based OSs are minimized. In some cases, when application tasks have short execution times and severe time constraints, the overhead introduced by the execution of basic RTOS functions cannot be neglected and can generate interference in task execution. In these situations, system predictability can only be guaranteed if the effects of OS overhead are taken into account in the feasibility analysis for establishing the scheduling scheme. In computer science, scheduling is the process by which processes are given access to system resources, e.g. processor cycles, communication bandwidth, and task priorities. HW_RTOS is a hardware operating system that achieves excellent real-time performance with low overhead compared to conventional software RTOS implementations. It supports mutexes, semaphores, time event indicators, and other standard APIs, enabling fast and consistent runtime and asynchronous external events response. This allows worst-case runtime to be specified, making it easier to design dynamic RTSs.

Modular implementation of the tsSMHwR + HwSA 4-pipeline stage RISC-V processor, a separate interface with the memory, design of the drivers for UART and I/O and mapping of the ports in the address space of the memory, as well as the legibility and explanation of the code, make the SoC project presented in this paper a flexible one, which can be easily tested and improved during further research activities. This paper presents and analyzes the results obtained from the synthesis, implementation, and testing of the tsSMHwR processor, using the Virtex-7 development kit. Through these tests carried out in hardware, it was intended to test, in practice, the theoretical elements presented in Section III, including the scheduling models and algorithms for RTS. Changing the task contexts is not based on saving the registers on the stack or saving in internal or external memory, but using the principle of remapping multiplexed resources. The contributions made by the authors in this paper consist of a feasibility study on the hardware scheduler HwSA_RTOS's implementation.

For this, an algorithm for mutex events has been presented, realized in hardware, and tested in FPGA. Thus, it was necessary to reorganize the HwSA unit and the control unit to meet the requirements imposed by the prioritized multiple event handling. For a better performance evaluation of the processor model described in this paper, the most representative implementations in the field were taken into account in the implementation and analysis of soft-core performance. The implementation of scheduling schemes in hardware and the significant reduction in context-switching times has led to a competitive and innovative architecture. At the same time, it was considered necessary, unlike other proposed implementations in which, in a multitasking system, there should be mechanisms for synchronization and inter-task communication. Implementing a CPU architecture without such mechanisms would be an unrealistic approach, i.e., a viable solution with practical applicability. In order to apply the proposed methodology, future research directions will consider the detailed presentation of the priority inheritance mechanism implemented in hardware at the HwSA_RTOS scheduler level, as well as the description of the test programs used for debugging and performance testing. It will also consider the presentation of application-level benchmark tests to demonstrate the real impact of reduced jitter and improved response times. The performance metrics obtained as quantitative results refer to the use of the open-license RISC-V ISA for the design of a 4-stage pipelined processor with integrated hardware scheduler and the reduction of FPGA resources used. Thus, the preemptive scheduler implements in hardware a mechanism for attaching to tasks (instPi) various types of events, such as time events, deadlines, asynchronous external events, mutexes, messages and signals for synchronization and intertask communication, thus implementing the priority inheritance mechanism, so that events inherit the priority of the task to which they have been assigned. Also HwSA_RTOS provides support for priority inversion when sharing resources by tasks through the instPi priority change mechanism via the mrPRIscPUi register.

PATENTS

The central processing unit with pipeline registers is patented in Germany, Munich (DE202012104250U1).

REFERENCES

- [1] E. Antolak and A. Pulka, "Energy-efficient task scheduling in design of multithread time predictable real-time systems," *IEEE Access*, vol. 9, pp. 121111–121127, 2021, doi: [10.1109/ACCESS.2021.3108912](https://doi.org/10.1109/ACCESS.2021.3108912).
- [2] S. A. Edwards and E. A. Lee, "The case for the precision timed (PRET) machine," in *Proc. 44th ACM/IEEE Design Autom. Conf.*, San Diego, CA, USA, Jun. 2007, pp. 264–265, doi: [10.1109/DAC.2007.375165](https://doi.org/10.1109/DAC.2007.375165).
- [3] R. Delgado and B. W. Choi, "New insights into the real-time performance of a multicore processor," *IEEE Access*, vol. 8, pp. 186199–186211, 2020.
- [4] V. G. Gaitan, N. C. Gaitan, and I. Ungurean, "CPU architecture based on a hardware scheduler and independent pipeline registers," *IEEE Trans. Very Large Scale Integr. (VLSI) Syst.*, vol. 23, no. 9, pp. 1661–1674, Sep. 2015.
- [5] E. Dodi and V. G. Gaitan, "Custom designed CPU architecture based on a hardware scheduler and independent pipeline registers—Concept and theory of operation," in *Proc. IEEE Int. Conf. Electron. Inf. Technol.*, Indianapolis, IN, USA, May 2012, pp. 1–5, doi: [10.1109/EIT.2012.6220705](https://doi.org/10.1109/EIT.2012.6220705).

- [6] N. Min-Allah, S. U. Khan, X. Wang, and A. Y. Zomaya, "Lowest priority first based feasibility analysis of real-time systems," *J. Parallel Distrib. Comput.*, vol. 73, no. 8, pp. 1066–1075, Apr. 2013.
- [7] S. Wu, S. Kumano, K. Marume, and M. Edahiro, "Task mapping and scheduling on RISC-V MIMD processor with vector accelerator using model-based parallelization," *IEEE Access*, vol. 12, pp. 35779–35795, 2024, doi: [10.1109/ACCESS.2024.3373902](https://doi.org/10.1109/ACCESS.2024.3373902).
- [8] R. Pan and G. Parmar, "SBIs: Application access to safe, baremetal interrupt latencies," in *Proc. IEEE 28th Real-Time Embedded Technol. Appl. Symp. (RTAS)*, May 2022, pp. 82–94, doi: [10.1109/RTAS54340.2022.00015](https://doi.org/10.1109/RTAS54340.2022.00015).
- [9] A. Gruin, T. Carle, H. Cassé, and C. Rochange, "Speculative execution and timing predictability in an open source RISC-V core," in *Proc. IEEE Real-Time Syst. Symp. (RTSS)*, Dec. 2021, pp. 393–404, doi: [10.1109/RTSS52674.2021.00043](https://doi.org/10.1109/RTSS52674.2021.00043).
- [10] A. Barengi, L. Breveglieri, N. Izzo, and G. Pelosi, "Exploring Cortex-M microarchitectural side channel information leakage," *IEEE Access*, vol. 9, pp. 156507–156527, 2021.
- [11] M. Reichenbach, J. Knödtel, S. Rachuj, and D. Fey, "RISC-v3: A RISC-V compatible CPU with a data path based on redundant number systems," *IEEE Access*, vol. 9, pp. 43684–43700, 2021.
- [12] S. M. Lee, E. N. Ri Ko, and S. E. Lee, "A hardware scheduler for multicore block cipher processor," in *Proc. 24th Euromicro Int. Conf. Parallel, Distrib., New-Based Process. (PDP)*, Heraklion, Greece, Feb. 2016, pp. 750–754.
- [13] S. Li, Y. Wei, and L. Ju, "Automatic data placement for CPU-FPGA heterogeneous multiprocessor system-on-chips," in *Proc. Design, Autom. Test Eur. Conf. Exhib. (DATE)*, Florence, Italy, Mar. 2019, pp. 1379–1384.
- [14] N. M. Qui, C. H. Lin, and P. Chen, "Design and implementation of a 256-bit RISC-V-based dynamically scheduled very long instruction word on FPGA," *IEEE Access*, vol. 8, pp. 172996–173007, 2020.
- [15] N. Gupta, S. Mandal, J. Malave, A. Mandal, and R. Mahapatra, "A hardware scheduler for real time multiprocessor system on chip," in *Proc. 23rd Int. Conf. VLSI Design*, Bengaluru, India, Jan. 2010, pp. 264–269.
- [16] Z. Zheng, X. Zhu, and H. Qian, "Design and implementation of arbitrary point FFT based on RISC-V SoC," in *Proc. IEEE 5th Adv. Inf. Technol., Electron. Autom. Control Conf. (IAEAC)*, Chongqing, China, Mar. 2021, pp. 1216–1219.
- [17] (2021). *Xilinx, MicroBlaze Soft Processor Core*. [Online]. Available: <https://www.xilinx.com/products/design-tools/microblaze.html>
- [18] J. Yiu, "System-on-chip design with arm cortex-m processors," Arm Educ. Media, Cambridge, U.K., 2019.
- [19] G. Ayers. (2014). *EXTensible Utah Multicore (XUM) Project*. Univ. Utah. [Online]. Available: <http://opencores.org/project,mips32r1>
- [20] *RPU*. Accessed: Sep. 2024. [Online]. Available: <https://github.com/Domipheus/RPU>
- [21] W.-T. Zhang, L.-F. Geng, D.-L. Zhang, G.-M. Du, M.-L. Gao, W. Zhang, N. Hou, and Y.-H. Tang, "Design of heterogeneous MPSoC on FPGA," in *Proc. 7th Int. Conf. ASIC*, Guilin, China, Oct. 2007, pp. 102–105.
- [22] *PicoRV32 A Size-optimized RISC-V CPU*. Accessed: Oct. 2024. [Online]. Available: <https://github.com/cliffordwolf/picorv32>
- [23] *NEORV32 Processor*. Accessed: Oct. 2024. [Online]. Available: <https://github.com/stnolting/neorv32>
- [24] *DarkRISCV*. Accessed: Oct. 2024. [Online]. Available: <https://github.com/darklife/darkriscv>
- [25] *UE RISC-V*. Accessed: Oct. 2024. [Online]. Available: <https://github.com/ultraembedded/riscv>
- [26] I. Zagan and V. G. Găitan, "Custom soft-core RISC processor validation based on real-time event handling scheduler FPGA implementation," *IEEE Access*, vol. 11, pp. 36264–36280, 2023, doi: [10.1109/ACCESS.2023.3266150](https://doi.org/10.1109/ACCESS.2023.3266150).
- [27] X. Li and X. He, "The improved EDF scheduling algorithm for embedded real-time system in the uncertain environment," in *Proc. 3rd Int. Conf. Adv. Comput. Theory Eng. (ICACTE)*, vol. 4, Aug. 2010, pp. 563–566.
- [28] K. Kotecha and A. Shah, "Adaptive scheduling algorithm for real-time operating system," in *Proc. IEEE Congr. Evol. Comput. (IEEE World Congr. Comput. Intell.)*, Jun. 2008, pp. 2109–2112.
- [29] M. D. Platzer and P. Puschner, "Vicuna: A timing-predictable RISC-V vector coprocessor for scalable parallel computation," in *Proc. 33rd Euromicro Conf. Real-Time Syst.*, 2021, pp. 1–16, doi: [10.4230/LIPIcs.ECRTS.2021.1](https://doi.org/10.4230/LIPIcs.ECRTS.2021.1).
- [30] L. Morais, C. Álvarez, D. Jiménez-González, J. M. de Haro, G. Araujo, M. Frank, A. Goldman, and X. Martorell, "Enabling HW-based task scheduling in large multicore architectures," *IEEE Trans. Comput.*, vol. 73, no. 1, pp. 138–151, Jan. 2024, doi: [10.1109/tc.2023.3323781](https://doi.org/10.1109/tc.2023.3323781).
- [31] L. Morais, V. Silva, A. Goldman, C. Alvarez, J. Bosch, M. Frank, and G. Araujo, "Adding tightly-integrated task scheduling acceleration to a RISC-V multi-core processor," in *Proc. 52nd Annu. IEEE/ACM Int. Symp. Microarchitecture*, Oct. 2019, pp. 861–872, doi: [10.1145/3352460.3358271](https://doi.org/10.1145/3352460.3358271).
- [32] A. Coluccio, U. Casale, A. Guastamacchia, G. Turvani, M. Vacca, M. R. Roch, M. Zamboni, and M. Graziano, "Hybrid-SIMD: A modular and reconfigurable approach to beyond von Neumann computing," *IEEE Trans. Comput.*, vol. 71, no. 9, pp. 2287–2299, Sep. 2022, doi: [10.1109/TPDS.2021.3127354](https://doi.org/10.1109/TPDS.2021.3127354).
- [33] D. Derafshi, A. Norollah, M. Khosroojam, and H. Beitollahi, "HRHS: A high-performance real-time hardware scheduler," *IEEE Trans. Parallel Distrib. Syst.*, vol. 31, no. 4, pp. 897–908, Apr. 2020, doi: [10.1109/TPDS.2019.2952136](https://doi.org/10.1109/TPDS.2019.2952136).
- [34] P. Kuacharoen, M. Shalan, and J. M. Vincent, "A configurable hardware scheduler for real-time systems," in *Proc. Int. Conf. Eng. Reconfigurable Syst. Algorithms*, Las Vegas, NV, USA, 2003, pp. 1–18.
- [35] *BiRISC-V*. Accessed: Jan. 2025. [Online]. Available: <https://github.com/ultraembedded/biriscv>
- [36] *URV Core*. Accessed: Jan. 2025. [Online]. Available: <https://github.com/twlostow/urv-core>



IONEL ZAGAN (Member, IEEE) received the M.Sc. and Ph.D. degrees from the Stefan cel Mare University of Suceava, Suceava, Romania.

He is currently an Associate Professor with the Department of Computers, Stefan cel Mare University of Suceava. His research interests include real-time systems, field-programmable gate arrays, microcontrollers, and pipeline processors with parallel execution of tasks. He is a member of the IEEE Computer Society.



VASILE GHEORGHITA GĂITAN (Member, IEEE) received the M.Sc. and Ph.D. degrees from the Gheorghe Asachi Technical University of Iasi, Iasi, Romania, in 1984 and 1997, respectively.

He is currently a Professor with the Department of Computers, Stefan cel Mare University of Suceava, Suceava, Romania. His research interests include real-time scheduling, embedded middleware, digital systems design with field-programmable gate arrays, fieldbuses, and

embedded system applications. He is a member of the IEEE Computer Society.

...